

Teach Module 21: API Observability and Monitoring

Module Focus

Module 21 covers API observability and monitoring for Java services.

Observability is how we understand what a running application is doing after it leaves our laptop. In production, you cannot debug by stepping through code. You need signals from the system itself.

The three core signals are:

1. Logs: What happened?
2. Metrics: How often, how much, how fast?
3. Traces: Where did the request travel, and where did it slow down or fail?

For Java and Spring services, observability usually means using tools like Spring Boot Actuator, Micrometer, Prometheus, Grafana, OpenTelemetry, and distributed tracing systems.

1. Why Observability Matters

Imagine you have a Java API endpoint:

```
@GetMapping("/orders/{id}")
public Order getOrder(@PathVariable Long id) {
    return orderService.findById(id);
}
```

In development, this looks simple. But in production, many things can go wrong:

- The database is slow.
- A downstream payment service is unavailable.
- Memory usage keeps increasing.
- Requests randomly time out.
- One customer reports failures, but others are fine.

Without observability, you only know "something broke."

With observability, you can answer:

- Which endpoint is failing?
- How many users are affected?
- When did it start?
- Is the API slow or is the database slow?
- Is this one instance or all instances?
- Did a recent deployment cause it?

That is the whole point.

2. Logs

Logs are event records. They describe things that happened inside the application.

Example:

```
private static final Logger logger = LoggerFactory.getLogger(OrderService.class);

public Order findById(Long id) {
    logger.info("Fetching order with id {}", id);

    return orderRepository.findById(id)
        .orElseThrow(() -> {
            logger.warn("Order not found: {}", id);
            return new OrderNotFoundException(id);
        });
}
```

Good logs are useful. Bad logs are noise.

Useful logs usually include:

- What operation happened
- Important IDs, like order ID or user ID
- Error details
- Enough context to investigate

Avoid logging sensitive data like passwords, tokens, credit card numbers, or personal information.

Log levels matter:

```
TRACE - extremely detailed debugging
DEBUG - developer-level detail
INFO  - normal application events
WARN  - something unexpected but not fatal
ERROR - something failed
```

In production, `INFO`, `WARN`, and `ERROR` are usually most important.

3. Metrics

Metrics are numeric measurements over time.

Examples:

```
HTTP requests per second
Average response time
Number of failed requests
Database connection pool usage
CPU usage
Memory usage
Queue length
```

Metrics help you see patterns.

For example:

```
/orders endpoint latency:  
10:00 AM - 120 ms  
10:05 AM - 180 ms  
10:10 AM - 900 ms  
10:15 AM - 2500 ms
```

This tells you something is degrading.

Counter

A counter is a value that only increases.

Example:

```
total_orders_created = 15200  
failed_login_attempts = 842  
http_requests_total = 991203
```

Use counters for "how many times did this happen?"

Gauge

A gauge is a value that can go up or down.

Example:

```
active_users = 120  
memory_used_mb = 760  
database_connections_active = 8
```

Use gauges for "what is the current value?"

Histogram

A histogram tracks distribution of values, usually timings or sizes.

Example:

```
request_duration_seconds
```

Histograms help answer:

- How many requests were under 100 ms?
- How many were between 100 ms and 500 ms?
- How many were over 1 second?

This is useful because averages can hide pain.

If 99 requests take 50 ms and 1 request takes 10 seconds, the average may not tell the real story. Percentiles are better:

```
p50 = normal user experience
p95 = slow user experience
p99 = worst-case experience
```

4. Spring Boot Actuator

Spring Boot Actuator exposes built-in operational endpoints.

Add the dependency:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Common endpoints:

```
/actuator/health
/actuator/info
/actuator/metrics
/actuator/env
/actuator/loggers
```

Example health response:

```
{
  "status": "UP"
}
```

You can expose selected endpoints in `application.properties` :

```
management.endpoints.web.exposure.include=health,info,metrics
management.endpoint.health.show-details=always
```

Important: do not expose everything publicly in production. Some actuator endpoints can reveal sensitive internals.

5. Health Checks

A health check tells whether the application is usable.

Basic example:

```
GET /actuator/health
```

Possible statuses:

```
UP
DOWN
```

```
OUT_OF_SERVICE
UNKNOWN
```

A health check can include dependencies:

- Database
- Message broker
- Disk space
- External service
- Cache

Example idea:

```
Application is running: yes
Database reachable: yes
Redis reachable: no
Disk space okay: yes
Overall status: DOWN
```

Kubernetes, Docker, load balancers, and cloud platforms often use health checks to decide whether traffic should be sent to a service.

6. Distributed Tracing

Modern applications often have multiple services.

Example request flow:

```
Frontend
-> Order API
  -> Inventory Service
  -> Payment Service
  -> Shipping Service
  -> Database
```

If the request takes 5 seconds, where was the delay?

Distributed tracing answers that.

A trace represents one full request journey.

A span represents one operation inside that journey.

Example:

```
Trace ID: abc123
```

```
Span 1: HTTP GET /checkout - 5200 ms
Span 2: call inventory-service - 80 ms
Span 3: call payment-service - 4900 ms
Span 4: save order to database - 120 ms
```

Now we know payment-service is the slow part.

Trace IDs are often added to logs too, so you can connect logs across services.

7. Dashboards

Dashboards visualize system behavior.

A useful API dashboard might show:

- Request rate
- Error rate
- Average latency
- p95 latency
- CPU usage
- Memory usage
- Database connections
- Top failing endpoints
- Recent deployments

A dashboard should answer operational questions quickly.

Bad dashboard:

```
50 charts nobody understands
```

Good dashboard:

```
Is the service healthy?  
Are users affected?  
Where is the problem likely located?  
Did this start after deployment?
```

8. Alerts

Alerts notify humans when something needs action.

Good alerts are actionable.

Bad alert:

```
CPU is 71%
```

Better alert:

```
Checkout API error rate is above 5% for 10 minutes
```

Common alert examples:

```
Error rate > 5%  
p95 latency > 2 seconds
```

```
Service health = DOWN
Database connections > 90%
Disk usage > 85%
No successful jobs in last 30 minutes
```

Avoid alert fatigue. If everything alerts, people start ignoring alerts.

9. The Four Golden Signals

A simple way to monitor APIs is with the four golden signals:

```
Latency
Traffic
Errors
Saturation
```

Latency: how long requests take.

Traffic: how many requests the system receives.

Errors: how many requests fail.

Saturation: how close the system is to its limits.

Examples of saturation:

```
CPU near 100%
Memory almost full
Thread pool exhausted
Database connection pool full
Queue length growing
```

If you remember only one monitoring model, remember this one.

10. Practical Java/Spring Example

Suppose you have this endpoint:

```
@GetMapping("/payments/{id}")
public Payment getPayment(@PathVariable Long id) {
    return paymentService.getPayment(id);
}
```

You should be able to observe:

```
How many times was /payments/{id} called?
How many calls failed?
How long did calls take?
Which downstream service was slow?
```

```
Was the app healthy during the failure?  
What error appeared in logs?
```

That is observability.

Mental Model

Think of observability like a hospital monitor for your application.

Logs are the notes.

Metrics are the vital signs.

Traces are the path through the body.

Health checks are the quick yes/no status.

Dashboards are the screen.

Alerts are the alarm.

A production Java engineer does not just write code that works locally. They write services that can be understood when things go wrong.

Mini Check

Answer these:

1. What is the difference between a log and a metric?
2. Why is p95 latency more useful than average latency?
3. What does Spring Boot Actuator provide?
4. What is the difference between a trace and a span?
5. Give one example of a bad alert and one example of a good alert.

Practice Exercises

Exercise 1: Add Actuator To A Spring Boot API

Create or use a small Spring Boot REST API and add:

```
spring-boot-starter-actuator
```

Then enable these endpoints:

```
management.endpoints.web.exposure.include=health,info,metrics  
management.endpoint.health.show-details=always
```

Practice checking:

```
/actuator/health  
/actuator/info  
/actuator/metrics
```


Goal: understand what Spring Boot can expose automatically.

Exercise 2: Create Custom Health Indicators

Add a custom health check that reports whether a fake external service is available.

Example behavior:

```
If fake service is available -> health is UP
If fake service is unavailable -> health is DOWN
```

Goal: learn how services report dependency health.

Exercise 3: Add Structured Logging

Create logs in a controller and service layer.

Log:

```
Request received
Business operation started
Business operation completed
Error occurred
```

Example:

```
logger.info("Fetching order with id={}", id);
logger.warn("Order not found id={}", id);
logger.error("Failed to process order id={}", id, exception);
```

Goal: practice useful logs without dumping sensitive data.

Exercise 4: Create Custom Metrics

Use Micrometer to create:

```
A counter for total orders created
A counter for failed order requests
A timer for order processing duration
A gauge for current active orders
```

Goal: learn the difference between counters, timers, and gauges.

Exercise 5: Measure Endpoint Latency

Create endpoints like:

```
GET /fast
GET /slow
GET /unstable
```

Make `/slow` sleep for 1-3 seconds.

Make `/unstable` randomly fail.

Then observe metrics through Actuator.

Goal: see latency and error metrics change in real time.

Exercise 6: Prometheus + Grafana Dashboard

Expose Prometheus metrics:

```
management.endpoints.web.exposure.include=health,metrics,prometheus
```

Add the Prometheus registry dependency, then connect Prometheus and Grafana.

Create dashboard panels for:

```
Request count
Error count
Average latency
p95 latency
JVM memory usage
CPU usage
```

Goal: turn raw metrics into visual monitoring.

Exercise 7: Add Alert Rules

Create simple alert rules such as:

```
Error rate > 5% for 5 minutes
p95 latency > 2 seconds for 10 minutes
Application health is DOWN
Memory usage > 85%
```

Goal: practice writing alerts that are actionable.

Exercise 8: Distributed Tracing Simulation

Create two small Spring Boot services:

```
order-service
payment-service
```

Have `order-service` call `payment-service`.

Add trace IDs to logs using OpenTelemetry or Micrometer tracing.

Then verify that one request can be followed across both services.

Goal: understand trace IDs, spans, and request flow.

Exercise 9: Failure Diagnosis Drill

Intentionally break something:

```
Stop the database
Make a downstream service return 500
Add a 5-second delay
Throw random exceptions
Exhaust a connection pool
```

Then use logs, metrics, health checks, and traces to answer:

```
What broke?
When did it start?
How many requests are affected?
Which component is responsible?
What should the alert say?
```

Goal: practice real production debugging.

Exercise 10: Build An Observability Checklist

For one API, document:

```
Important logs
Important metrics
Health checks
Dashboard panels
Alerts
Failure scenarios
```

Goal: think like an engineer responsible for production support.

Lab: API Observability And Monitoring

Lab Goal

Build a small Spring Boot API and add observability features:

```
Health checks
Actuator endpoints
Structured logs
Custom metrics
Slow/failing endpoints
Basic monitoring thinking
```

Scenario

You are building an `Order API`. The operations team wants to monitor whether the service is healthy, how many orders are created, how often requests fail, and which endpoints are slow.

Part 1: Create A Spring Boot Project

Create a Spring Boot app with these dependencies:

```
Spring Web
Spring Boot Actuator
Micrometer
```

If using Maven, make sure you have:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Part 2: Configure Actuator

In `application.properties` :

```
server.port=8080

management.endpoints.web.exposure.include=health,info,metrics
management.endpoint.health.show-details=always

info.app.name=Order Observability API
info.app.version=1.0.0
info.app.description=Practice API for observability and monitoring
```

Run the app and test:

```
http://localhost:8080/actuator/health
http://localhost:8080/actuator/info
http://localhost:8080/actuator/metrics
```

Part 3: Create An Order Controller

Create `OrderController.java` :

```
package com.example.observability;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.*;
import java.util.*;
```

```

import java.util.concurrent.ThreadLocalRandom;

@RestController
@RequestMapping("/orders")
public class OrderController {

    private static final Logger logger = LoggerFactory.getLogger(OrderController.class);

    private final Map<Long, String> orders = new HashMap<>();

    @PostMapping
    public String createOrder(@RequestParam String item) {
        long id = System.currentTimeMillis();
        orders.put(id, item);

        logger.info("Order created id={} item={}", id, item);

        return "Order created with id: " + id;
    }

    @GetMapping("/{id}")
    public String getOrder(@PathVariable Long id) {
        logger.info("Fetching order id={}", id);

        if (!orders.containsKey(id)) {
            logger.warn("Order not found id={}", id);
            return "Order not found";
        }

        return orders.get(id);
    }

    @GetMapping("/slow")
    public String slowEndpoint() throws InterruptedException {
        int delay = ThreadLocalRandom.current().nextInt(1000, 4000);
        logger.info("Slow endpoint called delayMs={}", delay);

        Thread.sleep(delay);

        return "Completed after " + delay + " ms";
    }

    @GetMapping("/unstable")
    public String unstableEndpoint() {
        boolean fail = ThreadLocalRandom.current().nextBoolean();

        if (fail) {
            logger.error("Unstable endpoint failed");
            throw new RuntimeException("Random failure occurred");
        }

        logger.info("Unstable endpoint succeeded");
    }
}

```

```
        return "Success";
    }
}
```

Test:

```
POST http://localhost:8080/orders?item=Laptop
GET http://localhost:8080/orders/123
GET http://localhost:8080/orders/slow
GET http://localhost:8080/orders/unstable
```

Part 4: Add Custom Metrics

Update the controller to use Micrometer:

```
import io.micrometer.core.instrument.Counter;
import io.micrometer.core.instrument.MeterRegistry;
```

Add fields:

```
private final Counter ordersCreatedCounter;
private final Counter orderFailuresCounter;
```

Add constructor:

```
public OrderController(MeterRegistry meterRegistry) {
    this.ordersCreatedCounter = Counter.builder("orders.created.total")
        .description("Total number of orders created")
        .register(meterRegistry);

    this.orderFailuresCounter = Counter.builder("orders.failures.total")
        .description("Total number of failed order operations")
        .register(meterRegistry);
}
```

Inside `createOrder()` :

```
ordersCreatedCounter.increment();
```

Inside the not found case:

```
orderFailuresCounter.increment();
```

Now check:

```
http://localhost:8080/actuator/metrics/orders.created.total
http://localhost:8080/actuator/metrics/orders.failures.total
```

Part 5: Add A Custom Health Indicator

Create `ExternalPaymentHealthIndicator.java` :

```
package com.example.observability;

import org.springframework.boot.actuate.health.Health;
import org.springframework.boot.actuate.health.HealthIndicator;
import org.springframework.stereotype.Component;
import java.util.concurrent.ThreadLocalRandom;

@Component
public class ExternalPaymentHealthIndicator implements HealthIndicator {

    @Override
    public Health health() {
        boolean paymentServiceAvailable = ThreadLocalRandom.current().nextInt(10) > 2;

        if (paymentServiceAvailable) {
            return Health.up()
                .withDetail("paymentService", "Available")
                .build();
        }

        return Health.down()
            .withDetail("paymentService", "Unavailable")
            .build();
    }
}
```

Test:

```
http://localhost:8080/actuator/health
```

Refresh a few times. Sometimes it should report `UP` , sometimes `DOWN` .

Part 6: Observe The Application

Perform these actions:

```
Create 3 orders
Search for 2 missing orders
Call /orders/slow 5 times
Call /orders/unstable 10 times
Check actuator health
Check actuator metrics
Review console logs
```

Answer these:

How many orders were created?
How many order lookup failures happened?
Which endpoint is slow?
What log messages helped you debug?
What would you alert on?

Part 7: Lab Deliverables

Submit or save:

1. Screenshot or output of `/actuator/health`
2. Screenshot or output of `/actuator/metrics`
3. Screenshot or output of custom order metrics
4. Console logs showing successful and failed requests
5. Short paragraph: What is the difference between logs, metrics, and health checks?

Bonus Challenge

Add Prometheus support:

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

Update:

```
management.endpoints.web.exposure.include=health,info,metrics,prometheus
```

Then test:

```
http://localhost:8080/actuator/prometheus
```

Look for:

```
orders_created_total
orders_failures_total
http_server_requests_seconds
```

This gives you a realistic first step toward production monitoring.